Evaluating the Mashgin Checkout System:

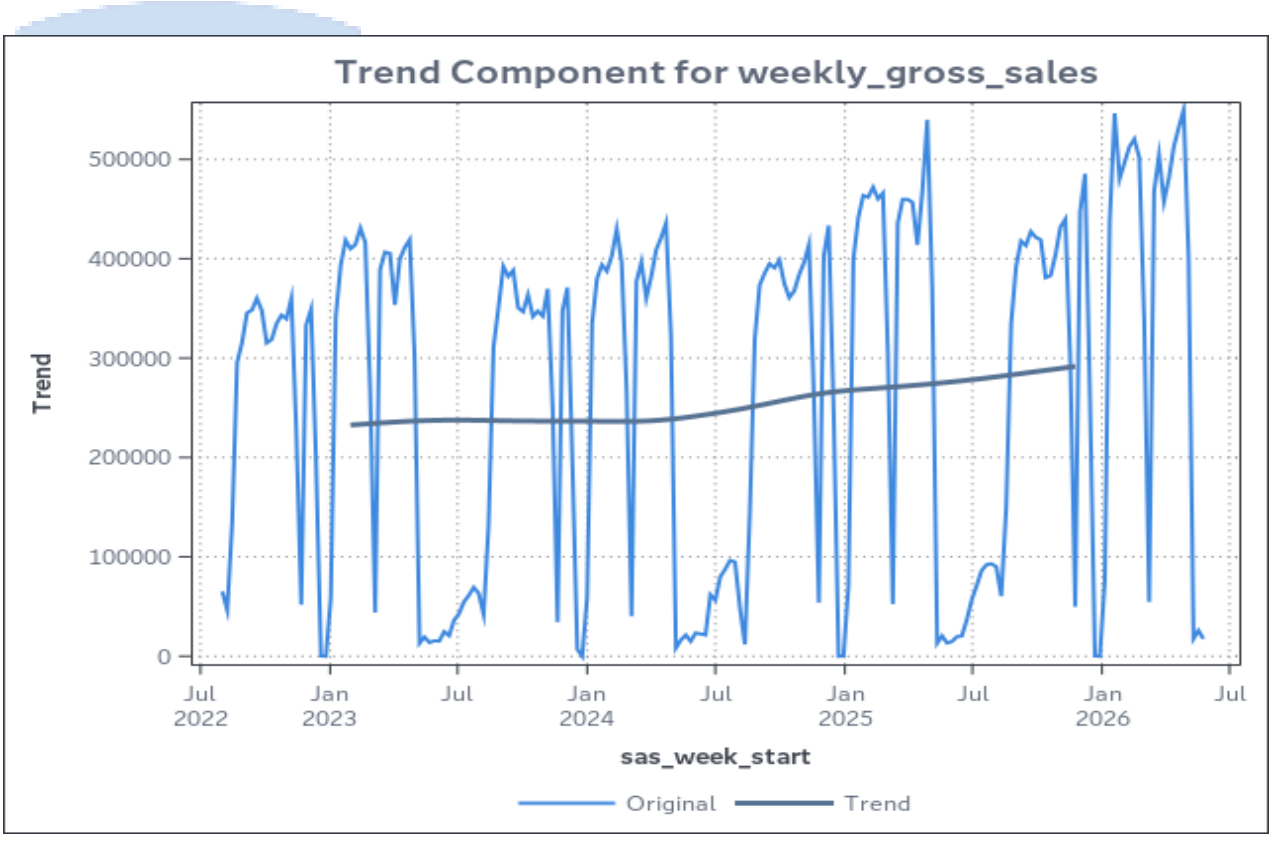
Steps for Modeling and Analysis

BAN 888: Implementing Analytics for Business

Penn State Smeal College of Business

Part II – Data Sourcing, Cleaning and Integration, Methodology Selection and Analytics Model Building

Aditi Jyoti Prakash Behera, Manoj Kuntamukkula, and Eric Matthew Smith



1. EXECUTIVE SUMMARY

In 2024, Penn State Auxiliary and Business Services began phasing Mashgin into its University Park convenience-store network. Mashgin is an AI-powered self-checkout system that scans multiple items at once, allowing customers to place items on the device instead of scanning each barcode individually. The project's main question is whether overall sales have improved during the Mashgin period after accounting for factors outside the checkout system, including price changes, population shifts, academic breaks, and store closures.

This phase focuses on preparing the data needed to answer that question fairly and on defining the modeling approach that will be used in the final analysis. The proposed primary model will use adjusted weekly sales as the main outcome, with transactions and basket size used as supporting measures to help explain the sales pattern. The data had to be pulled from legacy point-of-sale records, Mashgin API results, vendor dashboard checks, SQL staging tables, price indexes, population files, and academic calendar records. This section explains how those sources were cleaned, checked, and combined into one weekly store-level dataset so the proposed models can be built on consistent and auditable data.

2. LEGACY POINT OF SALE EXTRACTION

The history before Mashgin comes from the old MICROS point-of-sale system, which has records stored in SQL, totalling some 3,065,933 transactions for the stores between Aug. 2022 and July 2025. We aggregated weekly and set each row to track with the Monday that started its week, kept only the four convenience-store locations being compared, and totaled sales and transactions for each store, week by week. The result is a consistent weekly history we can regenerate at any time by rerunning the query.

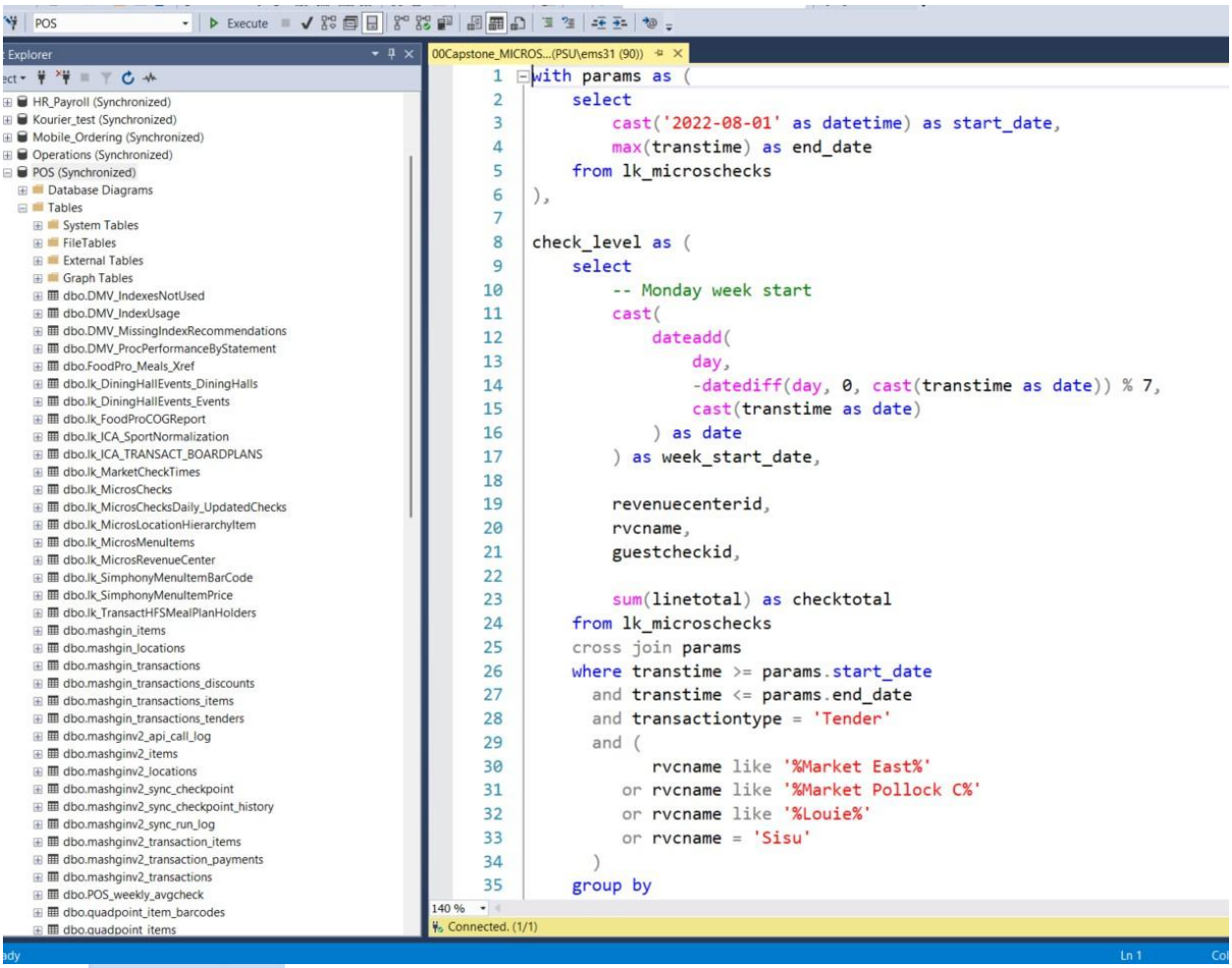


Figure 1. Legacy MICROS extraction logic showing Monday-start week calculation, location filters, and transaction-level aggregation into weekly measures.

3. MASHGIN EXTRACTION AND SOURCE VALIDATION

The Mashgin data, reflecting 3,953,932 transactions, needed more care due to the data coming from a temperamental API. The Mashgin API would perform slowly and cut off records even though it gave back success messages. A file feed is in the works, but in the meantime, the main source for individual transactional records is the API unless creating small exports from the Mashgin system. So, we checked the API against Mashgin's own reporting dashboard and the

raw data behind that dashboard, confirming that our totals matched what the vendor reported. This step matters because a data feed can run without errors and still have incomplete numbers. If that went unnoticed, the model could look precise while actually running on partial data.

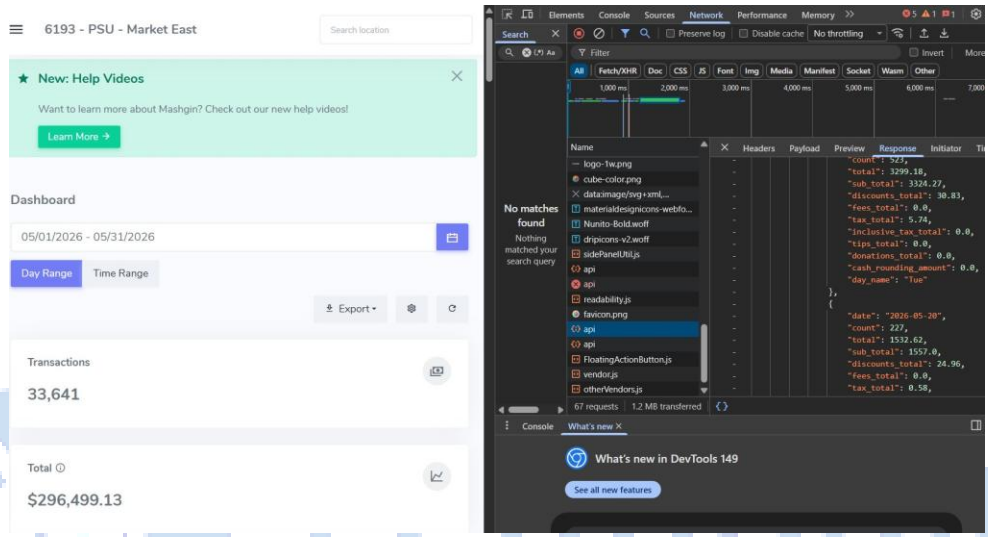


Figure 2. Mashgin dashboard and browser network response used to verify Mashgin API/staged sales and transaction results.

As you can see in Figure 2, the site results are built using the JSON data format behind the scenes. We were able to export monthly results into JSON text and send them to a database using a C# application. A more robust API was also created in C# that allows for throttling, record count validation, and retries. Example API and JSON import code is in Figure 3.

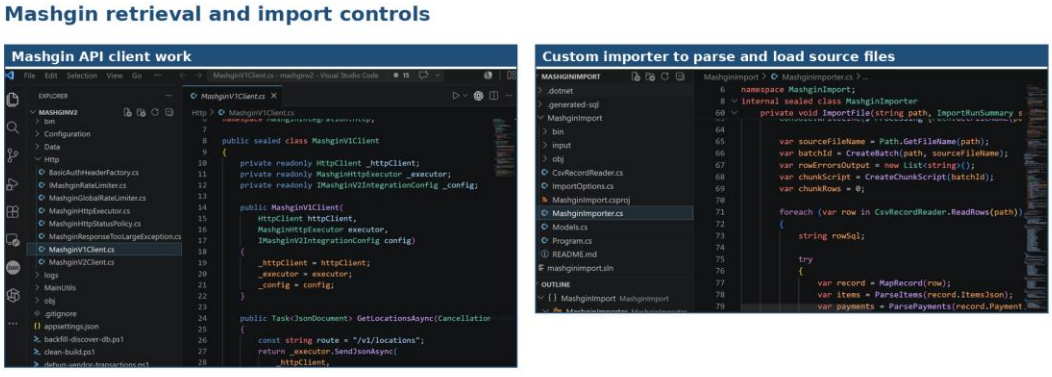


Figure 3. API-client and importer work showing how Mashgin source data was retrieved, parsed, and prepared for loading.

4. COMBINING RESULTS AND DATA SOURCE CONTROLS

The project used several data sources, each serving a different purpose in the modeling dataset. Because these sources came from different systems, each one required a specific cleaning or validation step before it could be used confidently. The table below summarizes each source, the control applied, and its value to the final analysis.

Table 1. Data Sources and Validation Controls

Data source	Role in the analysis	Validation applied	Value to the analysis
Legacy MICROS SQL	Historical sales, transactions, dates, and store names	Monday-start weeks; standardized location, tender, and check logic	Repeatable pre-Mashgin baseline
Mashgin API / vendor output	New-system sales and transaction records	Staged and checked before use as the analytical source	Traceable new-system dataset
Mashgin cloud reporting / JSON	Dashboard totals used for verification	Treated as a validation check, not the modeling source	Flags API pulls that do not reconcile to vendor totals
C# importer and SQL staging	Repeatable path from source files into SQL tables	Files parsed, loaded, reviewed, and staged before aggregation	Auditable pipeline rather than one-time exports
Matched item price file	Semester-level prices for comparable items	Prices matched across terms to build multipliers	Separates real performance from price increases
Population and academic calendar	Nearby population, breaks, and closure context	Multipliers, break indicators, and inactive-week flags added	Keeps population shifts and breaks out of the Mashgin read

5. ADDITION OF ADJUSTMENT VARIABLES

Raw sales figures alone can't answer the question, because two things changed over the study period that have nothing to do with Mashgin: prices went up, and the number of students living nearby shifted. To handle prices, we tracked a set of common items each semester and built a multiplier that puts every period in comparable terms. Price changes were tracked over time against the Spring 2026 prices for the multiplier. Generally, the prices went up as expected

due to inflation, although we note a period where they went down in Spring 2024 that warrants some follow-up with a subject matter expert. To handle population, we built a second multiplier from the nearby student count. Applying both gives a figure we call fully adjusted weekly sales, with sales with price and population differences stripped out, and that is the main number the analysis is built around, as mentioned in Table 1 and summarized in Table 2.

Table 2. Applied Price Multipliers by Semester

Term	Applied price multiplier	Weekly rows
Fall 2022	1.1427	83
Spring 2023	1.0819	100
Fall 2023	1.0351	79
Spring 2024	1.0257	120
Fall 2024	1.0294	108
Spring 2025	1.0351	105
Fall 2025	1.0000	83
Spring 2026	1.0000	80

The academic calendar matters too, because on a campus, there are several breaks and holidays. A normal week of classes, Spring Break, Thanksgiving, graduation, the gap between semesters, and summer shutdown all have impacts on sales and forecasting. We marked each of these in the data, along with how many days a location was actually open and a measure of how active the week was, which allowed us to help with better adjusted baselines for forecasting to reflect the percentage of time locations were open.

6. SUMMARY OF CLEANING STEPS COMPLETED

The cleaning process converted operational records from separate systems into a modeling dataset that could support a fair comparison of sales before and after Mashgin implementation. The main risk was that the model could mistake calendar effects, price changes,

and population changes for true system performance, and each cleaning step below was taken before ARIMAX models were built. Table 3 covers the data cleaning steps.

Table 3. Data Cleaning Steps Completed

Phase	Step	What was done	Why it matters
Sourcing	1	**Scoped the locations:** South, East, Pollock, and West; North excluded	Excludes non-comparable operations
	2	**Extracted legacy POS data:** MICROS SQL, completed tenders, Monday-start weeks	Builds the pre-Mashgin weekly baseline
	3	**Retrieved Mashgin results:** API and vendor sales and transaction records	Captures the post-Mashgin source, fields preserved
Validation	4	**Verified Mashgin totals:** dashboard and JSON checks against staged totals	Catches incomplete API pulls
	5	**Staged and parsed source files:** custom C# importer into SQL	Makes the import auditable and reloadable
Integration	6	**Standardized the weekly basis:** one row per store per week, IDs matched	One clean series per store, no double counting
	7	**Derived comparable metrics:** sales, transactions, average check, basket size	Consistent fields for pre/post comparison
Adjustment	8	**Built price and population controls:** semester-specific multipliers	Removes inflation and population change
	9	**Added academic-calendar controls:** breaks, holidays, active days and weeks	Accounts for campus operating intensity
	10	**Restricted to regular operating weeks:** open Spring and Fall semesters only	Excludes camp and training periods

The integrated dataset contains 715 store-week rows across South, East, Pollock, and West, spanning August 2022 through May 2026. Each row represents one store in one week and includes raw sales and transaction measures, system-period indicators, price and population multipliers, adjusted sales fields, and academic-calendar controls.

Table 4. Cleaned Data Summary and Quality Checks

Area checked	Result	Why it matters
Total cleaned rows	715 store-weeks	Enough weekly observations for time-series modeling

Area checked	Result	Why it matters
Stores included	South, East, Pollock, West	Comparable convenience-store operations only
Fields per row	24	Raw measures, adjustments, indicators, and calendar controls
Date range	Aug 2022 to May 2026	Covers both legacy and Mashgin periods
Legacy weeks	392	Substantial pre-Mashgin comparison period
Mashgin weeks	323	Substantial new-system comparison period
Negative sales or transactions	None found	No invalid operating values in the model
Zero-sales weeks	5	Holiday or shutdown weeks, not data errors
Zero-transaction weeks	3	Holiday weeks; average check not meaningful
Undefined average check	3 rows	Left blank when transactions equal zero
Price multiplier range	1.000 to 1.143	Reasonable semester-level price movement
Population multiplier range	0.725 to 1.217	Expected shifts in nearby student population
Blank open-semester adjusted fields	171 rows	Outside regular operation; intentionally not zero-filled
Full active weeks (7 days)	416 rows	Identifies normal operating weeks
Zero active-day weeks	171 rows	Identifies excluded or inactive periods
Modeled open-semester rows	544 store-weeks	Final modeling window after exclusions
Modeled weeks per store	136	Equal coverage for every location

The most important modeling decision was to leave nonregular operating weeks blank rather than forcing them to zero. These rows represent weeks outside the open-semester modeling window, including between-term periods, summer camp activity, training periods, and other nonstandard operations. Zero-filling those weeks would have made them look like real low-sales weeks and biased the sales comparison downward. By leaving them missing, the model focuses on normal store operations instead of treating closures or nonstandard weeks as poor performance.

7. MODELING READINESS AND METHODOLOGY SELECTION

The finished dataset gives the project one clean weekly table for modeling. Each store is measured on the same weekly basis, carries the same price, population, and calendar adjustments, and is limited to regular open-semester weeks so that the comparison reflects normal convenience-store operations. This structure is what the ARIMAX work requires: a consistent time series for each store, with nonstandard camp, training, summer, and between-term periods removed from the modeling window.

The primary modeling question is whether sales improved during the Mashgin period after price changes, closed dates, and population shifts were considered. For that reason, the headline dependent variable is fully adjusted weekly sales. Transactions and average basket size were also modeled, but they serve as supporting measures. Transactions help explain whether sales movement is connected to visit volume, while basket size helps show whether the value of each visit changed.

Time-series modeling was selected because the data is ordered by week and store performance follows a strong campus rhythm. A standard regression would treat each week as independent, even though sales in one week are related to recent weeks and to the academic calendar. SAS Viya's automatic forecasting process allowed ARIMA-family, ARIMAX or REGARIMA, and exponential-smoothing models to compete rather than forcing one model type onto every store. The final run used a 12-week holdout period for champion selection, so candidate models were compared against recent observations not used for fitting. This matters because a model can fit the historical data well but perform worse when tested against recent validation weeks.

Each store was modeled separately rather than pooled. East, Pollock, South, and West operate in different parts of campus and have different demand levels, so combining them would hide the store-level differences the analysis is trying to detect. In the main model, fully adjusted weekly sales is the dependent variable, the Monday week-start date is the time variable, and the system-period indicator is tested as an outside input using the 'use if significant' setting. Price and population multipliers are not reused as independent variables in the adjusted-sales model because they are already built into the dependent variable; including them again would double-count the same adjustment.

Table 5. Best-Fitting Model by Store and Measure

Measure	East	Pollock	South	West
Fully adjusted sales	RegARIMA	RegARIMA + legacy_ind	ESM	ARIMAX
Weekly transactions	RegARIMA	RegARIMA	ESM	ARIMAX
Average basket size	ARIMAX	ARIMAX	ESM	RegARIMA

Table 5 shows the main modeling result after the team standardized the 12-week holdout setting. For fully adjusted sales, East and West selected ARIMA-family models, South selected exponential smoothing, and Pollock was the only store where the system-period indicator was retained as an input. This means the evidence for a distinct system-period effect is still limited rather than uniform across all stores.

Adjusted transaction counts were modeled to test whether the Mashgin period changed visit volume after accounting for nearby population. The legacy/Mashgin indicator was not retained as a meaningful input, suggesting that transaction volume was better explained by recurring campus and seasonal patterns than by the checkout-system change itself.

Average basket size was modeled as the average number of items per transaction and was not adjusted for price, population, or active-week percentage. This measure is already normalized at the transaction level. The purpose of the basket-size model is to test whether the number of items purchased per visit changed when the checkout system changed from legacy POS to Mashgin. Therefore, the key question is whether the system-period indicator is retained as a meaningful input, not whether total volume changed because of population or store open percentage. And the initial answer is that we don't see a significant difference from the system change. There is no evidence that average basket size changed because of the switch from legacy checkout to Mashgin.

8. PRELIMINARY SALES MODEL INTERPRETATION

The fully adjusted sales models produced usable forecast accuracy for the store-level series. Because the official SAS Viya run used a 12-week holdout, WAPE is the most practical accuracy metric here. WAPE stands for weighted absolute percentage error. It is calculated as total absolute forecast error divided by total actual sales. Using the SAS summary output, this is equivalent to MAE divided by average actual fully adjusted weekly sales. In the official run, WAPE ranged from 8.4% to 16.4%, meaning total absolute forecast error was about 8 to 16 percent of total actual fully adjusted sales. R-square is shown only as supporting context.

Table 6. Fully Adjusted Weekly Sales Model Fit

Store	Selected model	WAPE	MAE	Forecast fit
South	ESM	8.4%	$7,530	**Strong**
Pollock	RegARIMA + legacy_ind	12.7%	$9,811	**Moderate**
West	ARIMAX	13.1%	$6,318	**Moderate**
East	RegARIMA	16.4%	$25,170	**Fair**

The results suggest that store performance is explained mostly by store-specific weekly and seasonal patterns, with one location retaining the system-period indicator. East and West selected ARIMA-family models without the system input, and South selected exponential smoothing. Pollock is the exception: its selected RegARIMA model retained legacy_ind as an input. That does not prove a systemwide Mashgin sales effect, but it does identify Pollock as the location that needs the most careful follow-up.

The modeling process is repeatable. The same weekly input file can be regenerated from the documented extraction, cleaning, adjustment, and filtering steps. SAS Viya then produces standard output tables for each measure, including parameter estimates, fit statistics, model specifications, and forecasts. Tables 5 and 6 are built from the standardized 12-week holdout run, so another analyst using the same cleaned input series and modeling setup should be able to reproduce the champion models and the conclusion that the system indicator was retained only in the Pollock adjusted-sales model.

9. INITIAL FINDINGS

The first modeling results suggest that the system-period effect is more limited than a simple before-and-after comparison might imply. Across the modeled sales results, the legacy and Mashgin indicator was retained in only one place: Pollock's adjusted sales model. East and West selected ARIMA-family models and South selected exponential smoothing, but those models did not retain the system indicator. The supporting transaction and basket-size models did not retain the system indicator either.

Table 7. Initial Modeling Findings

Area	Initial finding	Business meaning	Signal
Sales	System input retained at Pollock only; other stores selected ARIMA-family or ESM models	Evidence is store-specific, not systemwide	**Isolated**
Visits	System indicator not retained in the transaction models	Visits follow weekly and seasonal demand	**No effect**
Basket size	System indicator not significant	No clear system effect on items per visit	**No effect**
Model accuracy	WAPE of 8.4% to 16.4% on the 12-week holdout	Errors are small enough for practical planning use, although some additional trimming of dates to account for ramp-up times may help.	**Usable**

The main takeaway is not that Mashgin failed, and it is also not that Mashgin clearly improved sales across all stores. The better interpretation is that the current evidence does not support a broad systemwide claim. Pollock is the only location where the system-period indicator appears in the selected adjusted-sales model, so that result should be treated as a follow-up question rather than proof of a Mashgin effect.

10. MARKET-BASKET ANALYSIS FOLLOW-UP

Additionally, we are going to follow up with market-basket analysis that potentially addresses how ABS could raise the value of each visit. The first completed run used April 2026 Mashgin item-line data. The raw file contained 555,885 scanned item rows. After removing non-merchandise or unusable lines and de-duplicating repeated instances of the same item within a transaction, the analysis retained 514,379 transaction-item rows, representing 298,344 unique transactions and 4,174 distinct item names. The average transaction contained 1.72 unique items, and the median transaction contained one unique item, which confirms that many convenience-store visits are small, quick purchases.

The basket results in Table 8 are exploratory, but they are already useful because they point to practical merchandising actions. If Mashgin makes checkout faster, ABS still needs ways to improve the value of each visit. Product pairings such as cereal with milk, chips with dip, and yogurt with fruit give managers concrete ideas for layout, signage, and bundle testing.

Initial Apriori findings suggest that ABS can use basket analysis to identify obvious add-on bundles such as cereal with milk, chips with dip, and yogurt with fruit. Second, it can guide store layout by placing complementary products within easy reach of each other. Third, it can distinguish true cross-category bundle opportunities from same-family variety behavior.

Table 8. Initial Market-Basket Findings

Finding area	Early result	Operational use	Strength
Cereal and milk	Cereal purchases strongly associated with milk	Nearby placement, signage, and add-on promotion	**Strong**
Chips and dip	Salty snacks strongly associated with dips and spreads	Bundle testing and endcap or party displays	**Strong**
Yogurt and fruit	Yogurt associated with bananas and other fruit	Healthier add-on placement; breakfast pairing	**Moderate**
Prepared meals and snacks	Some prepared foods paired with snack items	Targeted meal-deal testing over broad promotion	**Moderate**
Same-family variety	Similar products or flavors bought together	Shelf arrangement; weak case for cross-category bundles	**Weak**

11. PROJECT NEXT STEPS

The project remains on schedule. Since Part I, the team has completed the major data preparation work, built a comparable weekly store-level dataset, run the first round of time-series models, and completed an initial market-basket analysis. The early results are useful because they are narrowing the final recommendation. At this point, the evidence does not point to a simple systemwide sales lift from Mashgin. Instead, the results suggest that performance differs

by location, with one store showing a signal that needs additional review before it is treated as a business conclusion.

Table 9. Planned Next Steps by Focus Area

Focus area	Next step	Why it matters	Owner
Data and modeling	Smooth start- and end-of-semester ramp weeks that distort charts and forecasts	Removes outlier weeks that skew the trend and predictions	**Analysis team**
	Re-run Pollock without the population adjustment	Tests whether the population control is driving the Pollock result	**Analysis team**
Pricing	Verify with the SME why prices dipped one semester before rising again	Confirms the observed price change is real, not a data issue	**Consult ABS SME**
Store-level questions	Review why the Pollock adjustments may not affect C-store traffic	Explains the one store where the system indicator was retained	**Both**
	Confirm East growth tracks the rising nearby student population	Validates the population driver behind East's trend	**Consult ABS SME**
	Confirm West upperclassmen remain a steady, unchanging group	Supports treating West demand as stable in the model	**Consult ABS SME**
Open business questions	Review staffing savings and tradeoffs under Mashgin	Adds a cost dimension the analysis has not yet covered	**Consult ABS SME**
	Review any other Mashgin issues and open input from ABS on Mashgin	Surfaces operational problems not visible in sales data	**Consult ABS SME**
Recommendations	Research bundling and other merchandising practices	Turns the basket findings into actions ABS can test	**Analysis team**

Owner key: **Analysis team** = the project team runs it. **Consult ABS SME** = confirm with the Auxiliary and Business Services subject-matter expert. **Both** = the team investigates and confirms with the SME.

The remaining work is focused and within the original project scope. The team will review the model results, test the Pollock finding more carefully, and translate the sales, transaction, and basket-size results into business terms. In parallel, the market-basket results will be used to develop practical sales-improvement recommendations, such as product placement, signage, and bundle opportunities based on items that customers already tend to buy together.

The final deliverable will bring these pieces together: where the sales models show a reliable signal, where they do not, what the Pollock result means after further review, and what merchandising actions ABS can test based on the basket analysis. And we'll provide industry recommendations for market basket and layout ideas, as well as collaborate with a subject matter expert from ABS. Overall, we're planning to have a well-researched, informed, and collaborative

document in hopes that it is helpful when looking at vendor impacts and for considering similar projects in the future.

APPENDIX

Distribution of Sales

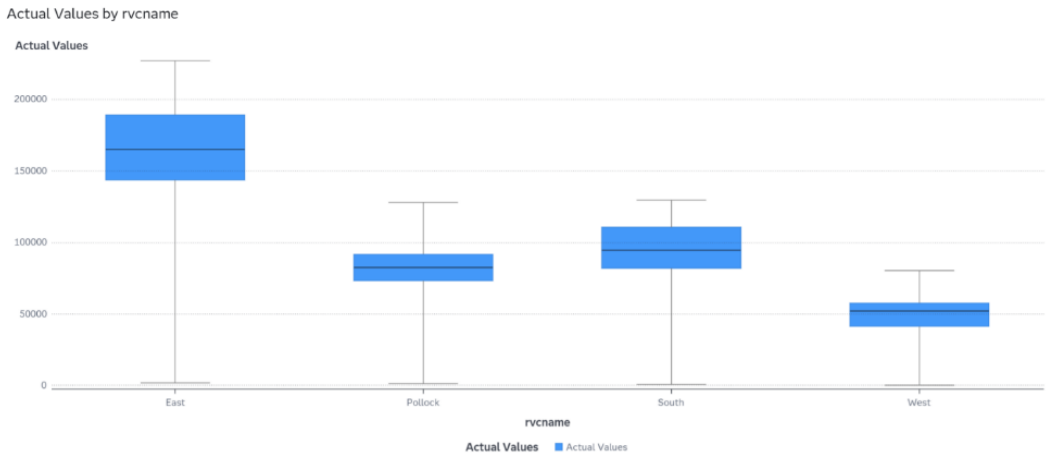


Figure 4. Box plots showing the distribution of adjusted sales amounts during regular semester open times.

Average Basket Size

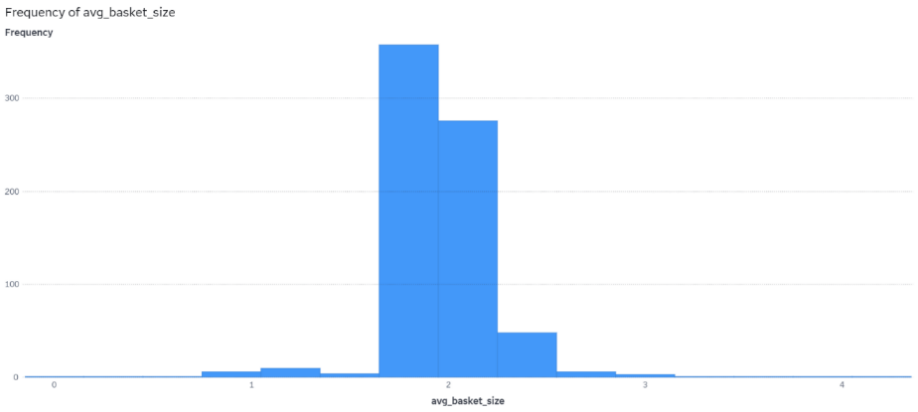


Figure 5. Frequency distribution of average basket size – averages close to 2 items per sale.

Average Basket Size Over Time

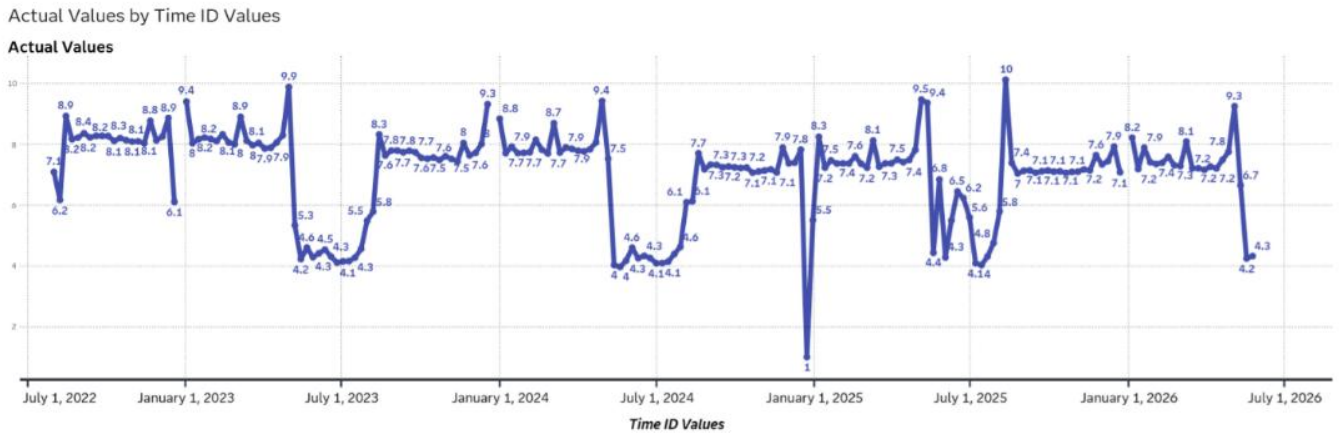


Figure 6. Average basket size over time, which illustrates that the ramp-up times could use cleanup to help with model performance.

Population-Adjusted Transactions

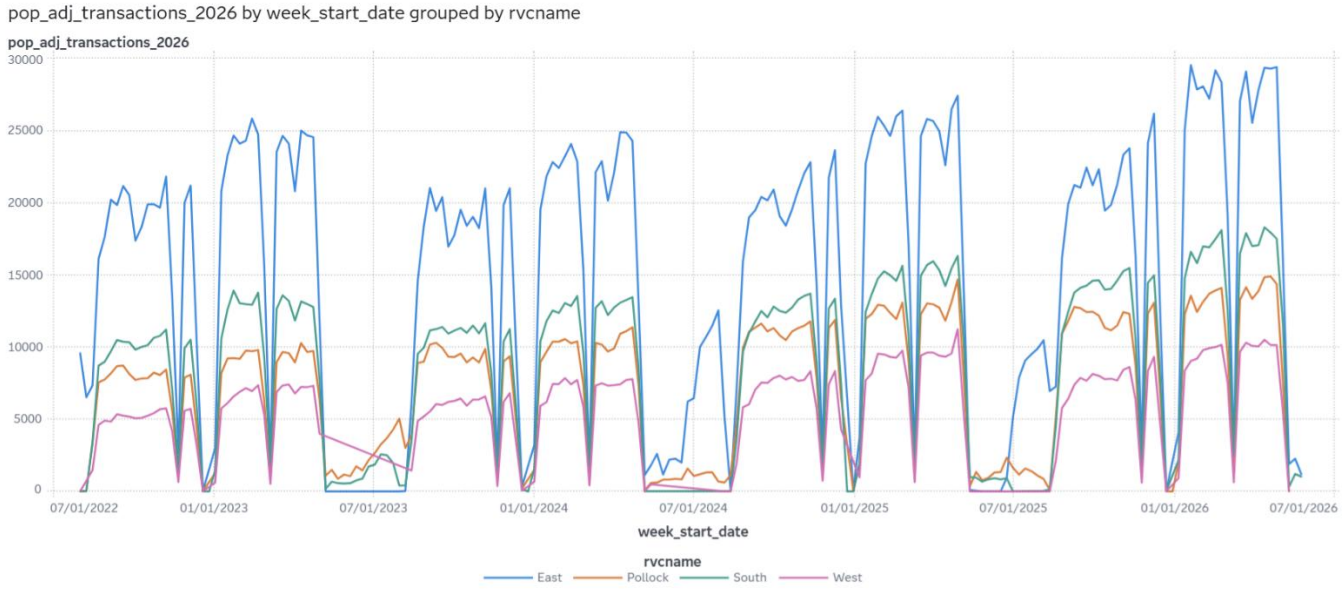


Figure 7. Population-adjusted transactions factor in changes in the nearby dorm population.

Fully Adjusted Sales SAS Forecast Preliminary View

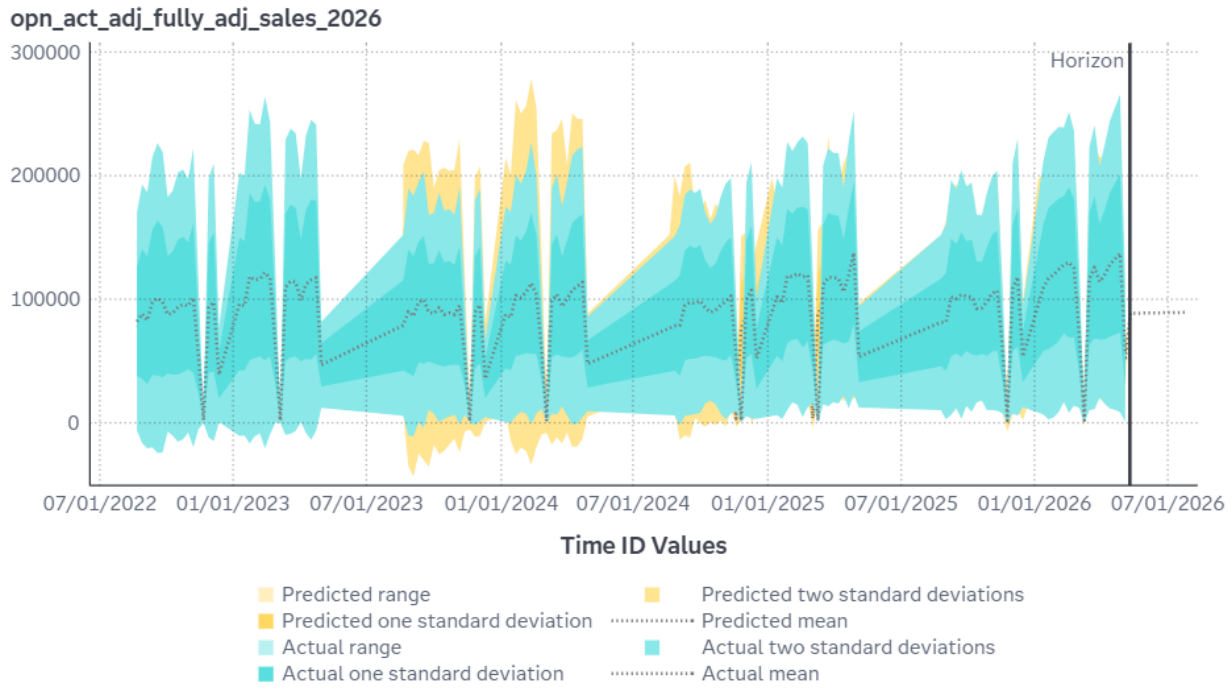


Figure 8. The fully adjusted sales model in SAS Viya, accounting for price and population changes and regular semester open times, shows that sales remain steady over time, both before Mashgin and after implementation.

Python Code for Apriori Market Basket Analysis

```
"""
Mashgin Strategic Bundle Recommendation Analysis - fast CSV version
====================================================================

This script is designed for your Mashgin item-level export and is more useful
for business recommendations than a raw Apriori/lift-ranked table.

It creates CSV files for:
    1. category_addon_themes.csv        X category -> Y category
    2. category_third_item_themes.csv   X category + Y category -> Z category
    3. strategic_cross_category_skus.csv item X -> item Y across categories
    4. same_family_variety_skus.csv     flavor/size variety rules
    5. same_category_cross_brand_skus.csv same category but different family/brand
    6. item_support_metadata.csv        item counts with inferred category/family
    7. basket_size_distribution.csv     how many unique items per basket
    8. readme_summary.csv               key metrics and settings

Install:
    pip install pandas openpyxl numpy

Run from the same folder as the Excel file:
    python mashgin_strategic_bundle_fast_csv.py
"""

# Import libraries used for transaction grouping, combinations, and Excel/CSV processing.
from __future__ import annotations

from collections import Counter
from itertools import combinations
from pathlib import Path
import math
```

```
import re
from typing import Iterable

import numpy as np
import pandas as pd

# Configure the source workbook, worksheet, and output directory.
INPUT_XLSX = Path("2026-04_Mashgin_Items.xlsx")
SHEET_NAME = "in"
OUTPUT_DIR = Path("mashgin_strategic_outputs")

# Apply minimum support, confidence, and lift thresholds to retain actionable rules.
MIN_ITEM_SUPPORT_COUNT = 100
MIN_ITEM_RULE_SUPPORT_COUNT = 20
MIN_ITEM_CONFIDENCE = 0.015
MIN_ITEM_LIFT = 1.05
MAX_ITEMS_PER_BASKET_FOR_COMBOS = 20

MIN_CATEGORY_RULE_SUPPORT_COUNT = 300
MIN_CATEGORY_CONFIDENCE = 0.03
MIN_CATEGORY_LIFT = 1.05
MIN_CATEGORY_TRIPLE_SUPPORT_COUNT = 100
MIN_CATEGORY_TRIPLE_CONFIDENCE = 0.04
MIN_CATEGORY_TRIPLE_LIFT = 1.05

# Suppress nonstrategic categories and remove non-merchandise transaction lines.
SUPPRESS_BUSINESS_CATEGORIES = {"Other", "Personal Care"}
REMOVE_ITEM_KEYWORDS = ["TAX", "REFUND", "VOID", "DISCOUNT", "BAG FEE"]


# Standardize item text and format itemsets for readable output.
def clean_text(value: object) -> str:
    if pd.isna(value):
        return ""
    return re.sub(r"\s+", " ", str(value).strip())


def itemset_to_text(values: Iterable[str]) -> str:
    return " + ".join(sorted(str(x) for x in values))


# Assign each item to a business category using keyword-based rules.
def infer_category(item_name: str) -> str:
    s = item_name.lower()
    if any(k in s for k in ["toothpaste", "toothbrush", "mouthwash", "listerine", "deodorant", "shampoo", "conditioner", "soap",
"tampon", " pads", "advil", "tylenol", "medicine", "bandage"]):
        return "Personal Care"
    if any(k in s for k in ["frosted flakes", "cinnamon toast crunch", "lucky charms", "cereal", "cheerios", "mini wheats",
"raisin bran", "froot loops", "captain crunch", "cocoa puffs", "trix", "gm ", "kellogg"]):
        return "Cereal/Breakfast"
    if any(k in s for k in ["banana", "blueberr", "raspberr", "blackberr", "strawberr", "grapes", "apple", "orange", "clementine",
"pineapple", "fruit cup", "watermelon", "solely fruit", "mango", "cut fruit"]):
        return "Fruit"
    if any(k in s for k in ["life water", "aquafina", "smartwater", "smart water", "essentia", "fiji", "dasani", "water 20 oz",
"water 1 liter", "water 700 ml", "water bottle", "water gallon"]):
        return "Water"
    if any(k in s for k in ["alani", "red bull", "monster", "celsius", "rockstar", "prime energy", "bucked up", "ghost energy",
"energy drink"]):
        return "Energy Drink"
    if any(k in s for k in ["pepsi", "coke", "coca", "sprite", "dr pepper", "mountain dew", "mtn dew", "soda", "poppi", "starry",
"fanta", "root beer", "ginger ale", "7 up"]):
        return "Soda/Functional Soda"
    if any(k in s for k in ["propel", "gatorade", "powerade", "bodyarmor", "body armor"]):
        return "Sports Drink"
    if any(k in s for k in ["juice", "lemonade", "tea", "coffee", "latte", "frappuccino", "kombucha", "cold brew", "iced tea"]):
        return "Juice/Tea/Coffee"
    if any(k in s for k in ["milk", "creamery chug", "choc recovery"]):
        return "Milk/Dairy Drink"
    if any(k in s for k in ["yogurt", "oikos", "chobani", "clio", "siggi", "activia"]):
        return "Yogurt"
    if any(k in s for k in ["ice cream", "mochi", "popsicle", "frozen", "scoots", "gelato", "icee", "klondike", "novelty"]):
        return "Frozen Dessert"
    if any(k in s for k in ["cookie", "brownie", "brookie", "muffin", "donut", "doughnut", "bakery", "cake", "pastry", "cinnamon
roll", "rice krispie treat"]):
        return "Bakery/Dessert"
    if any(k in s for k in ["chips", "doritos", "tostitos", "cheetos", "fritos", "lays", "lay's", "ruffles", "pretzel", "popcorn",
"cheez it", "cheez-it", "goldfish", "takis", "pringles", "sunchips", "sun chips", "smartfood", "fl "]):
        return "Salty Snack"
    if any(k in s for k in ["candy", "chocolate", "rees", "hershey", "snicker", "m&m", "skittle", "starburst", "gummy", "sour
patch", "twix", "kit kat", "gum", "mint", "nerds", "airheads", "swedish fish", "jolly rancher"]):
        return "Candy/Gum"
    if any(k in s for k in ["protein", "quest", "clif", "kind bar", "granola", "nature valley", "rxbar", "perfect bar",
"barebells", "built", "think bar", "power crunch"]):
        return "Nutrition/Bar"
    if any(k in s for k in ["cheese", "sargento", "string cheese", "cheddar", "colby", "pepper jack", "mozzarella"]):
        return "Cheese"
    if any(k in s for k in ["dip", "hummus", "salsa", "guacamole", "buffalo chix", "queso"]):
        return "Dip/Spread"
    if any(k in s for k in ["sandwich", "wrap", "salad", "sushi", "burrito", "meal", "lunchable", "soup", "maruchan", "ramen",
"noodle", "mac and cheese", "pizza", "grilled cheese", "panera"]):
        return "Meal/Prepared"
```

```
    if any(k in s for k in ["cracker", "crackers"]):
        return "Crackers"
    return "Other"


# Maintain known product families for brand- and family-level rule classification.
KNOWN_FAMILIES = [
    "Oikos Triple Zero", "Creamery Chug", "Nature Valley", "Cinnamon Toast Crunch", "Frosted Flakes", "Lucky Charms", "Alani Nu",
"My Mochi", "Sargento", "Life Water", "Aquafina", "Propel", "Poppi", "Creamery", "Scoots", "Bar Clio", "Clio", "Tostitos",
"Doritos", "Cheetos", "Fritos", "Lay's", "Lays", "Ruffles", "Cheez It", "Cheez-It", "Goldfish", "Quest", "Clif", "Kind",
"Reese's", "Reeses", "Hershey", "Snickers", "M&M", "Skittles", "Starburst", "Dr Pepper", "Pepsi", "Coke", "Coca Cola", "Mountain
Dew", "Gatorade", "Powerade", "Bodyarmor", "Red Bull", "Celsius", "Monster", "Prime", "Maruchan", "Good Humor", "Demets Flipz",
"BH", "Chobani", "Barebells", "Built", "Kellogg", "GM", "Panera", "Pierre", "Heluva Good", "Solely", "Listerine", "Crest",
"Kodiak", "Jif"
]


# Infer a product family from known labels or a fallback two-token item name.
def infer_family(item_name: str) -> str:
    s = item_name.lower()
    for family in sorted(KNOWN_FAMILIES, key=len, reverse=True):
        if family.lower() in s:
            return family
    for family in ["Bananas", "Blueberries", "Raspberries", "Blackberries", "Strawberries", "Grapes", "Apples", "Oranges",
"Pineapple", "Mango"]:
        if family.lower().rstrip("s") in s:
            return family
    stop = {"ss", "oz", "ml", "liter", "litre", "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "12", "16", "20", "24", "1/2",
"pint", "ct", "count"}
    tokens = [t for t in re.split(r"\s+", item_name) if t.lower() not in stop]
    return " ".join(tokens[:2]) if tokens else item_name


# Convert transaction-item rows into unique item or category baskets.
def grouped_unique_lists_fast(df: pd.DataFrame, tx_col: str, value_col: str) -> list[list[str]]:
    baskets: dict[str, set[str]] = {}
    for tx, value in df[[tx_col, value_col]].dropna().itertuples(index=False, name=None):
        baskets.setdefault(str(tx), set()).add(str(value))
    return [sorted(v) for v in baskets.values()]


# Read the export, clean transaction lines, remove duplicates, and attach metadata.
def read_and_clean() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    raw = pd.read_excel(
        INPUT_XLSX,
        sheet_name=SHEET_NAME,
        header=None,
        names=["basket_number", "transaction_id", "basket_code", "local_time", "item_name"],
        engine="openpyxl",
    )
    raw["transaction_id"] = raw["transaction_id"].astype(str).str.strip()
    raw["item_name"] = raw["item_name"].map(clean_text)
    tx_parts = raw["transaction_id"].str.split("-", expand=True)
    raw["location_code"] = tx_parts[1] if tx_parts.shape[1] > 1 else np.nan

    cleaned = raw.dropna(subset=["transaction_id", "item_name"]).copy()
    cleaned = cleaned[cleaned["item_name"] != ""]
    pattern = "|".join(re.escape(x) for x in REMOVE_ITEM_KEYWORDS)
    cleaned = cleaned[~cleaned["item_name"].str.contains(pattern, case=False, na=False)]
    cleaned = cleaned.drop_duplicates(["transaction_id", "item_name"])

    n_tx = cleaned["transaction_id"].nunique()
    item_counts = cleaned.groupby("item_name")["transaction_id"].nunique().sort_values(ascending=False)
    metadata = item_counts.rename("transaction_count").reset_index()
    metadata["support"] = metadata["transaction_count"] / n_tx
    metadata["category"] = metadata["item_name"].map(infer_category)
    metadata["family"] = metadata["item_name"].map(infer_family)

    cleaned = cleaned.merge(metadata[["item_name", "category", "family"]], on="item_name", how="left")
    return raw, cleaned, metadata


# Produce descriptive metrics that document the analysis population.
def make_summary(raw: pd.DataFrame, cleaned: pd.DataFrame, metadata: pd.DataFrame) -> pd.DataFrame:
    basket_sizes = cleaned["transaction_id"].value_counts()
    rows = [
        ["Input rows", len(raw)],
        ["Cleaned transaction-item rows", len(cleaned)],
        ["Unique transactions", cleaned["transaction_id"].nunique()],
        ["Unique item names", cleaned["item_name"].nunique()],
        ["Average unique items per transaction", round(float(basket_sizes.mean()), 3)],
        ["Median unique items per transaction", round(float(basket_sizes.median()), 3)],
        ["Transactions with 1 unique item", int((basket_sizes == 1).sum())],
        ["Transactions with 2+ unique items", int((basket_sizes >= 2).sum())],
        ["Transactions with 3+ unique items", int((basket_sizes >= 3).sum())],
        ["Business note", "Use Category Addon Themes and Strategic Cross Category SKUs first; treat Same Family Variety as a
separate shelf/variety insight."],
    ]
    return pd.DataFrame(rows, columns=["metric", "value"])
```

```
# Summarize the number of unique items contained in each transaction.
def basket_distribution(cleaned: pd.DataFrame) -> pd.DataFrame:
    sizes = cleaned["transaction_id"].value_counts()
    out = sizes.value_counts().sort_index().rename_axis("unique_items_in_transaction").reset_index(name="transaction_count")
    out["transaction_percent"] = out["transaction_count"] / out["transaction_count"].sum()
    return out


# Generate category-level two-item and three-item association rules.
def category_rules(cleaned: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    n_tx = cleaned["transaction_id"].nunique()
    work = cleaned[~cleaned["category"].isin(SUPPRESS_BUSINESS_CATEGORIES)]
    baskets = grouped_unique_lists_fast(work, "transaction_id", "category")
    cat_counts = Counter(); pair_counts = Counter(); triple_counts = Counter()
    for cats in baskets:
        cat_counts.update(cats)
        if len(cats) >= 2:
            pair_counts.update(combinations(cats, 2))
        if len(cats) >= 3:
            triple_counts.update(combinations(cats, 3))

    # Evaluate directional category rules (X -> Y) using support, confidence, and lift.
    addon_rows = []
    for (a, b), count in pair_counts.items():
        if count < MIN_CATEGORY_RULE_SUPPORT_COUNT:
            continue
        for ant, con in [(a, b), (b, a)]:
            ant_count = cat_counts[ant]; con_count = cat_counts[con]
            conf = count / ant_count; con_support = con_count / n_tx
            lift = conf / con_support if con_support else np.nan
            expected = ant_count * con_count / n_tx
            inc = count - expected
            if conf >= MIN_CATEGORY_CONFIDENCE and lift >= MIN_CATEGORY_LIFT:
                addon_rows.append({
                    "theme": f"{ant} -> {con}",
                    "offer_idea": f"Connect {ant} with {con} through placement or a small add-on deal.",
                    "plain_english": f"Among baskets with {ant}, {conf:.1%} also include {con}; {lift:.2f}x baseline; about
{inc:.0f} baskets above random expectation.",
                    "antecedent_category": ant, "consequent_category": con,
                    "support_count": count, "antecedent_count_tx": ant_count, "consequent_count_tx": con_count,
                    "confidence": conf, "lift": lift, "expected_count_if_random": expected,
                    "incremental_baskets_vs_random": inc,
                    "strategic_score": max(inc, 0) * conf * math.log1p(count),
                })

    # Evaluate two-category antecedents that predict a third category.
    triple_rows = []
    for (a, b, c), count in triple_counts.items():
        if count < MIN_CATEGORY_TRIPLE_SUPPORT_COUNT:
            continue
        for pair, con in [((a, b), c), ((a, c), b), ((b, c), a)]:
            pair = tuple(sorted(pair))
            pair_count = pair_counts[pair]; con_count = cat_counts[con]
            conf = count / pair_count; con_support = con_count / n_tx
            lift = conf / con_support if con_support else np.nan
            expected = pair_count * con_count / n_tx
            inc = count - expected
            if conf >= MIN_CATEGORY_TRIPLE_CONFIDENCE and lift >= MIN_CATEGORY_TRIPLE_LIFT:
                ant_text = itemset_to_text(pair)
                triple_rows.append({
                    "theme": f"{ant_text} -> {con}",
                    "offer_idea": f"When a basket has {ant_text}, test a deal or placement that adds {con}.",
                    "plain_english": f"Among baskets with {ant_text}, {conf:.1%} also include {con}; {lift:.2f}x baseline; about
{inc:.0f} baskets above random expectation.",
                    "antecedent_categories": ant_text, "consequent_category": con,
                    "support_count": count, "antecedent_count_tx": pair_count, "consequent_count_tx": con_count,
                    "confidence": conf, "lift": lift, "expected_count_if_random": expected,
                    "incremental_baskets_vs_random": inc,
                    "strategic_score": max(inc, 0) * conf * math.log1p(count),
                })

    addon = pd.DataFrame(addon_rows).sort_values(["strategic_score", "incremental_baskets_vs_random"], ascending=False) if
addon_rows else pd.DataFrame()
    triple = pd.DataFrame(triple_rows).sort_values(["strategic_score", "incremental_baskets_vs_random"], ascending=False) if
triple_rows else pd.DataFrame()
    return addon, triple


# Generate SKU-level rules after pruning items with insufficient support.
def item_rules(cleaned: pd.DataFrame) -> pd.DataFrame:
    n_tx = cleaned["transaction_id"].nunique()
    keep = cleaned.groupby("item_name")["transaction_id"].nunique()
    keep_items = keep[keep >= MIN_ITEM_SUPPORT_COUNT].index
    work = cleaned[cleaned["item_name"].isin(keep_items)]
    baskets = grouped_unique_lists_fast(work, "transaction_id", "item_name")
    meta = work[["item_name", "category", "family"]].drop_duplicates("item_name").set_index("item_name")

    # Count item support and pair co-occurrence; skip unusually large baskets for efficiency.
    item_counts = Counter(); pair_counts = Counter(); skipped = 0
    for items in baskets:
        item_counts.update(items)
```

```
        if len(items) > MAX_ITEMS_PER_BASKET_FOR_COMBOS:
            skipped += 1
            continue
        if len(items) >= 2:
            pair_counts.update(combinations(items, 2))

    # Calculate rule metrics and classify each retained rule by business use.
    rows = []
    for (a, b), count in pair_counts.items():
        if count < MIN_ITEM_RULE_SUPPORT_COUNT:
            continue
        for ant, con in [(a, b), (b, a)]:
            ant_count = item_counts[ant]; con_count = item_counts[con]
            conf = count / ant_count; con_support = con_count / n_tx
            lift = conf / con_support if con_support else np.nan
            if conf < MIN_ITEM_CONFIDENCE or lift < MIN_ITEM_LIFT:
                continue
            expected = ant_count * con_count / n_tx
            inc = count - expected
            ant_cat = meta.loc[ant, "category"]; con_cat = meta.loc[con, "category"]
            ant_family = meta.loc[ant, "family"]; con_family = meta.loc[con, "family"]
            same_cat = ant_cat == con_cat; same_family = ant_family == con_family
            suppressed = ant_cat in SUPPRESS_BUSINESS_CATEGORIES or con_cat in SUPPRESS_BUSINESS_CATEGORIES
            if same_family:
                klass = "Same family variety"; weight = 0.5
            elif not same_cat and not suppressed:
                klass = "Strategic cross-category"; weight = 3.0
            elif same_cat and not same_family and not suppressed:
                klass = "Same category cross-brand"; weight = 1.25
            elif not same_cat:
                klass = "Cross-category but review"; weight = 1.0
            else:
                klass = "Low-priority"; weight = 0.5
            rows.append({
                "recommendation_class": klass,
                "bundle_text": f"If basket has [{ant}], suggest [{con}]",
                "offer_idea": f"Test add-on placement or a small deal: {ant} + {con}.",
                "plain_english": f"Among baskets with {ant}, {conf:.1%} also include {con}; {lift:.2f}x baseline; about {inc:.1f}
baskets above random expectation.",
                "antecedent": ant, "consequent": con, "target_bundle": itemset_to_text([ant, con]),
                "antecedent_category": ant_cat, "consequent_category": con_cat,
                "antecedent_family": ant_family, "consequent_family": con_family,
                "same_category": same_cat, "same_family": same_family,
                "support_count": count, "antecedent_count_tx": ant_count, "consequent_count_tx": con_count,
                "confidence": conf, "lift": lift, "expected_count_if_random": expected,
                "incremental_baskets_vs_random": inc,
                "strategic_score": max(inc, 0) * conf * math.log1p(count) * weight,
                "skipped_large_baskets": skipped,
            })
    return pd.DataFrame(rows)


# Write each nonempty result table to the configured output directory.
def save_csv(df: pd.DataFrame, name: str) -> None:
    if df is not None and not df.empty:
        df.to_csv(OUTPUT_DIR / name, index=False)


# Run the complete workflow and save all business-facing CSV outputs.
def main() -> None:
    OUTPUT_DIR.mkdir(exist_ok=True)
    print("Reading and cleaning...")
    raw, cleaned, metadata = read_and_clean()
    print(f"Transactions: {cleaned['transaction_id'].nunique():,}; unique items: {cleaned['item_name'].nunique():,}")

    print("Building category themes...")
    cat_addons, cat_triples = category_rules(cleaned)

    print("Building SKU rules...")
    sku_rules = item_rules(cleaned)
    sku_rules = sku_rules.sort_values(["recommendation_class", "strategic_score", "support_count"], ascending=[True, False,
False]) if not sku_rules.empty else sku_rules

    print("Saving CSV outputs...")
    save_csv(make_summary(raw, cleaned, metadata), "readme_summary.csv")
    save_csv(basket_distribution(cleaned), "basket_size_distribution.csv")
    save_csv(metadata, "item_support_metadata.csv")
    save_csv(cat_addons, "category_addon_themes.csv")
    save_csv(cat_triples, "category_third_item_themes.csv")
    if not sku_rules.empty:
        save_csv(sku_rules[sku_rules["recommendation_class"] == "Strategic cross-category"].sort_values(["strategic_score",
"incremental_baskets_vs_random"], ascending=False), "strategic_cross_category_skus.csv")
        save_csv(sku_rules[sku_rules["recommendation_class"] == "Same family variety"].sort_values(["support_count",
"confidence"], ascending=False), "same_family_variety_skus.csv")
        save_csv(sku_rules[sku_rules["recommendation_class"] == "Same category cross-brand"].sort_values(["strategic_score",
"support_count"], ascending=False), "same_category_cross_brand_skus.csv")
        save_csv(sku_rules, "all_sku_rules.csv")

    print(f"Done. CSV files saved in: {OUTPUT_DIR.resolve()}")
```